Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

10. Chuyển đổi thực tế — từ Monolith đến Microservices . . . 2
10.1. Thời điểm thích hợp để chuyển đổi . . . 3
10.1.1. Vấn đề: microservices không phải đích đến mặc định . . . 3
10.1.2. Ma trận quyết định — khi nào cân nhắc chuyển đổi . . . 3
10.1.3. KBM: kiến trúc tiến hóa (Evolutionary Architecture), không phải Big Bang . . . 3
10.1.4. Bài học thực tế — khi chuyển đổi đi sai hướng (anti-pattern) . . . 4
10.2. Strangler Fig Pattern — chuyển đổi tăng dần . . . 5
10.2.1. Vấn đề: chuyển đổi tức thì ("Big Bang") so với chuyển đổi tăng dần (incremental migration) . . . 5
10.2.2. Strangler Fig Pattern . . . 5
10.2.3. API Gateway — điểm tiếp nối (seam) cho Strangler Fig . . . 6
10.2.4. KBM: Strangler Fig thực tế — từ `kbm-legacy-monolith` đến `kbm-*` . . . 7
10.3. Tách database — thách thức lớn . . . 8
10.3.1. Vấn đề: database coupling nguy hiểm hơn code coupling . . . 8
10.3.2. Cạm bẫy ghi kép (dual-write) và Outbox Pattern . . . 9
10.3.3. CDC cho chuyển đổi dữ liệu . . . 11
10.4. Anti-Corruption Layer và các pattern chuyển đổi . . . 11
10.4.1. Anti-Corruption Layer (ACL) — bảo vệ ranh giới . . . 11
10.4.2. Branch by Abstraction . . . 12
10.4.3. Parallel Run — kiểm chứng trước khi chuyển đổi . . . 13
10.5. Quá trình chuyển đổi kiến trúc tại KBM . . . 14
10.5.1. Tổng hợp các phân tích khoảng trống . . . 14
10.5.2. Lộ trình chuyển đổi — 4 giai đoạn . . . 15
10.5.3. Ma trận quyết định — ưu tiên theo tác động × nỗ lực . . . 17
10.6. Sai lầm thường gặp khi chuyển đổi . . . 18
10.7. Tổng kết . . . 18
10.8. Đọc thêm . . . 19
Tài liệu tham khảo . . . 20

CHƯƠNG 10

Chuyển đổi thực tế — từ Monolith đến Microservices

> "If you can't build a well-structured monolith, what makes you think microservices is the answer?"
> — Simon Brown

Lưu ý

Chuyển đổi (Migration) là một *kỹ năng riêng*: áp dụng các pattern đã học ở Phần I–II vào một hệ thống *đang vận hành thực tế*, nơi không thể viết lại từ đầu. Chương này thêm yếu tố *phán đoán* — khi nào nên và khi nào *không* nên chuyển sang microservices — và là nơi trình bày **lộ trình chuyển đổi chi tiết cho KBM** (Quick Wins → Observability → Resilience → Database Decomposition).

Mục tiêu: chuyển đổi nền tảng LMS mà không làm gián đoạn việc học. Chương này giúp người đọc:

MỤC TIÊU HỌC TẬP

- Hiểu khi nào nên và **khi nào không nên** chuyển sang microservices
- Nắm vững **Strangler Fig Pattern** — chiến lược chuyển đổi tăng dần phổ biến nhất
- Áp dụng **5 chiến lược tách database** từ góc nhìn chuyển đổi thực tế
- Hiểu **Anti-Corruption Layer** và các pattern chuyển đổi bảo vệ ranh giới hệ thống
- Thiết kế **lộ trình chuyển đổi** khả thi, ưu tiên theo tác động và nỗ lực (Impact × Effort)
- Phân tích lộ trình cho KBM, từ khoảng trống ở Chương 2–9 đến điều kiện tiên quyết ở Chương 11–12

10.1. Thời điểm thích hợp để chuyển đổi

10.1.1. Vấn đề: microservices không phải đích đến mặc định

Sau 9 chương phân tích các pattern kiến trúc, cơ chế giao tiếp, quản lý dữ liệu, API Gateway và bảo mật, độc giả có thể có ấn tượng rằng microservices là kiến trúc mọi hệ thống nên hướng tới. **Đây là một sai lầm phổ biến.**

Martin Fowler — người đồng tác giả bài viết gốc "Microservices" (2014) — cảnh báo rõ ràng [W2]:

Monolith First

> *"Almost all the cases where I've heard of a system that was built as a microservice system from scratch, it has ended up in serious trouble... You shouldn't start a new project with microservices, even if you're sure your application will be big enough to make it worthwhile."*
>
> *— Martin Fowler [W2]*

Newman trong [4b, Ch.1] bổ sung: microservices là **một lựa chọn kiến trúc** không phải bước tiến hóa bắt buộc. Monolith không phải "kiến trúc sai" — monolith là kiến trúc đúng cho phần lớn hệ thống ở giai đoạn đầu.

10.1.2. Ma trận quyết định — khi nào cân nhắc chuyển đổi

Bảng 10.1 tóm tắt các tiêu chí giúp quyết định khi nào monolith vẫn đủ tốt và khi nào nên cân nhắc microservices.

Bảng 10.1: *Ma trận quyết định (Decision Matrix) — khi nào cân nhắc chuyển đổi*

Tiêu chí	Monolith đủ tốt	Cân nhắc Microservices
Quy mô đội ngũ	Ước lượng ≤ 8 người	Ước lượng > 15 người, nhiều nhóm
Tần suất triển khai	Hàng tuần/tháng	Triển khai hàng ngày theo nhóm
Yêu cầu mở rộng	Mở rộng toàn bộ là đủ	Mở rộng từng thành phần độc lập
Đa dạng công nghệ	Một bộ công nghệ (tech stack) là đủ	Cần đa ngôn ngữ (polyglot: Java + Python + Go)
Độ phức tạp nghiệp vụ	Ranh giới chưa rõ	Bounded context đã xác định rõ (Chương 2)
Mức độ trưởng thành của tổ chức	Chưa có DevOps, CI/CD	Đã có CI/CD, container hóa, giám sát

Martin Fowler gọi đây là **Microservice Premium** — chi phí phải trả khi chuyển sang microservices: độ phức tạp của hệ phân tán, lỗi mạng, eventual consistency, chi phí vận hành. Khoản phí này (premium) chỉ đáng trả khi lợi ích (như triển khai độc lập, mở rộng linh hoạt, sự tự chủ của đội ngũ) vượt qua chi phí.

10.1.3. KBM: kiến trúc tiến hóa (Evolutionary Architecture), không phải Big Bang

KBM không bắt đầu từ một bản thiết kế microservices hoàn chỉnh định sẵn. Hệ thống đã đi qua nhiều thế hệ: các judge monolith riêng cho bài lập trình, Network Judge, SQL Judge mới, rồi dần hình thành các service cho auth, academic, judge, notification và DevOps Lab. Vì vậy câu hỏi đúng không phải là "có nên microservices-first không?", mà là: *mỗi bước tiến hóa có đang giải quyết đúng vấn đề của thời điểm đó không?*

Microservice Premium trong KBM

KBM chọn hướng microservices vì hai lý do cụ thể: (1) mục đích **giáo dục** — bản thân hệ thống là tài liệu học tập cho sinh viên, (2) bảy core bounded context đã đủ rõ sau nhiều năm vận hành các hệ thống chấm điểm (judge system): Identity & Access, Academic Management, SQL Practice, Code Practice, Network Practice, DevOps Practice và Examination & Proctoring.

Tuy nhiên, sự đánh đổi (trade-off) hiện rõ: (1) cơ sở dữ liệu dùng chung (shared database) giữa Core Service và Academic Service — sự phụ thuộc (coupling) vẫn tồn tại dù dịch vụ đã được tách, (2) thư viện dùng chung (shared library) chứa quá nhiều logic — rủi ro nguyên khối phân tán (distributed monolith), (3) môi trường đa ngôn ngữ (polyglot) Java+Go làm tăng chi phí biên dịch, kiểm thử và triển khai. Đây là ví dụ thực tế của Microservice Premium: **lợi ích giáo dục và nhu cầu cô lập hệ thống chấm điểm đủ để chứng minh tính hợp lý của chi phí**, nhưng vẫn cần một lộ trình giảm thiểu sự phụ thuộc (coupling) có kiểm soát.

Sự khác biệt giữa các phân hệ chấm điểm cũng minh họa rõ việc chọn chiến lược theo từng ngữ cảnh. Network Practice được thiết kế độc lập để đánh giá giao tiếp qua mạng nên tách rất tự nhiên theo giao thức. Trong khi đó, với SQL Practice, bài toán nằm ở ranh giới giữa bộ điều phối `kbm-judge-sql` và các worker `kbm-sandbox-*` chạy trong môi trường cô lập theo từng hệ quản trị cơ sở dữ liệu (`kbm-sandbox-mysql`, `kbm-sandbox-postgres`, `kbm-sandbox-sqlserver`, ...). Chuyển đổi đúng hướng là làm rõ cơ chế định tuyến theo quyền sở hữu (ownership routing), chuẩn hóa hợp đồng công việc và kết quả, bổ sung idempotency dựa trên `judgeRunId`, sau đó mới mở rộng theo chiều ngang cho từng nhóm worker dựa trên tải thực tế của từng hệ quản trị cơ sở dữ liệu.

Monolith First — tổng hợp

Bắt đầu bằng monolith. Chỉ chuyển sang microservices khi monolith thực sự trở thành vấn đề và đã biết mình đang giải quyết vấn đề nào. Những hệ thống microservices tệ nhất thường do các nhóm bắt đầu bằng microservices *trước khi* hiểu ranh giới domain.

— Tổng hợp từ Martin Fowler [W2] và Sam Newman [4b, Ch.1]

Monolith First không phủ nhận những gì đã học — mà là nguyên tắc lựa chọn thời điểm và cách áp dụng đúng ngữ cảnh.

10.1.4. Bài học thực tế — khi chuyển đổi đi sai hướng (anti-pattern)

Dù quyết định chuyển đổi là đúng đắn, *cách thức* chuyển đổi vẫn có thể gây thất bại cho dự án. Richardson trong [2b, Ch.2] phân tích bốn anti-pattern phổ biến nhất khi chuyển từ monolith sang microservices — chúng ta minh họa qua các sai lầm *tiềm năng* nếu chia tách sai trong KBM:

Bảng 10.2: *Các anti-pattern khi chuyển đổi sang microservices (minh họa với KBM)*

Anti-pattern	Ví dụ sai lầm tiềm năng trong KBM	Hậu quả
Data Service *(Dịch vụ bao bọc CRUD)*	Tách riêng bảng `questions` thành một `QuestionDataService` chỉ làm CRUD — Core Service phải gọi HTTP mỗi lần cần đọc câu hỏi	Tăng độ trễ mạng, mất lợi ích giao dịch (transaction) nội bộ
Fine-grained Service *(Chia quá nhỏ)*	Chia Auth thành 3 service: `UserProfile`, `UserCredential`, `UserSession`	Mỗi lần đăng nhập phải điều phối (orchestrate) 3 dịch vụ, khó gỡ lỗi (debug)

Anti-pattern	Ví dụ sai lầm tiềm năng trong KBM	Hậu quả
Microservices-first	Chia KBM thành nhiều service ngay từ đầu, khi ranh giới miền chưa rõ, nên tách theo tầng kỹ thuật (technical layer) — mỗi thay đổi chấm điểm ảnh hưởng nhiều service cùng lúc	Phát sinh "Distributed Monolith" (xem thêm §10.6)
QA Gate đầu cuối *(Chờ kiểm thử tích hợp toàn cục)*	Triển khai độc lập nhưng đội kiểm thử (QA) phải chờ tích hợp tất cả để kiểm thử	Triệt tiêu khả năng triển khai độc lập *(Independent Deployability)*

Các sai lầm chuyển đổi khác (Big Bang Rewrite, Lift-and-Shift, Over-engineering) sẽ được phân tích chi tiết tại §10.6.

10.2. Strangler Fig Pattern — chuyển đổi tăng dần

10.2.1. Vấn đề: chuyển đổi tức thì ("Big Bang") so với chuyển đổi tăng dần (incremental migration)

Có hai cách thức chính để dịch chuyển từ monolith sang microservices: **Big Bang** (viết lại toàn bộ hệ thống mới và chuyển đổi cùng lúc) mang rủi ro rất cao và thường dẫn đến thất bại; hoặc **Tăng dần (Strangler Fig)** (tách dần từng phần, cho chạy song song) với rủi ro thấp hơn nhiều do có thể dễ dàng hoàn tác (rollback) từng phần.

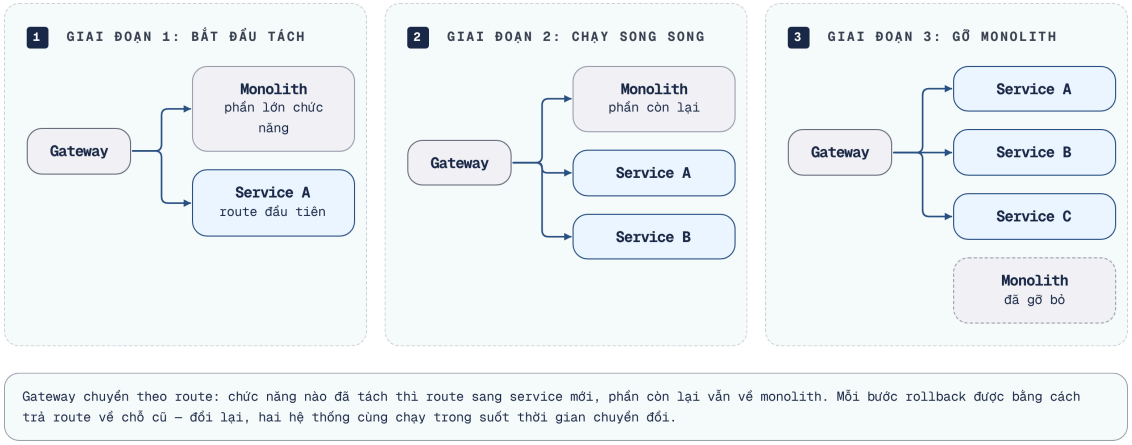
Newman trong [4b, Ch.2] và Richardson trong [2a, Ch.13] đều cảnh báo: **viết lại kiểu Big Bang rất rủi ro và thường thất bại**. Khi viết lại từ con số không, đội ngũ phát triển mất toàn bộ kiến thức nghiệp vụ (domain knowledge) được nhúng trong mã nguồn cũ, thời hạn (deadline) liên tục trễ, và khách hàng phải chờ đợi *tất cả* tính năng trước khi nhận *bất kỳ* giá trị nào.

10.2.2. Strangler Fig Pattern

Xây thư viện số hiện đại

Hãy tưởng tượng trường đại học muốn thay thư viện vật lý cũ bằng thư viện số. Tòa nhà cũ không bị đập bỏ ngay lập tức (Big Bang); thay vào đó, thư viện số được xây ngay bên cạnh (Strangler Fig). Ban đầu, khi sinh viên đến cổng trường (API Gateway) để xin "Mượn sách điện tử", bảo vệ chỉ họ sang tòa nhà mới. Nếu họ xin "Mượn sách giấy", bảo vệ vẫn chỉ vào tòa nhà cũ. Từng khu vực (tạp chí, giáo trình...) lần lượt được số hóa và sinh viên được chuyển hướng sang tòa nhà mới, cho đến khi tòa nhà cũ không còn ai vào nữa và bị phá dỡ hoàn toàn.

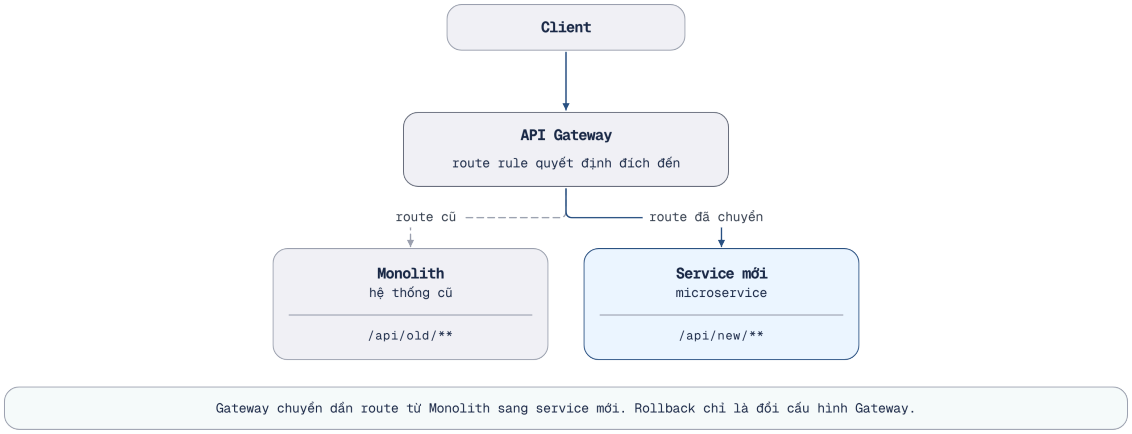
Strangler Fig — pattern do Martin Fowler đặt tên (2004), lấy cảm hứng từ cây đa bóp nghẹt (strangler fig) bao quanh cây chủ, dần thay thế cho đến khi cây chủ biến mất [W8].



Hình 10.1: *Strangler Fig Pattern — dần thay thế monolith qua 3 giai đoạn*

10.2.3. API Gateway — điểm tiếp nối (seam) cho Strangler Fig

Để áp dụng mô hình này, chúng ta cần khả năng định tuyến giao tiếp tốt. API Gateway (đã học ở Chương 8) hoạt động như một điểm tiếp nối (seam) cho Strangler Fig, giúp che giấu sự phức tạp của quá trình chuyển đổi khỏi người dùng cuối:



Hình 10.2: *API Gateway làm điểm tiếp nối cho Strangler Fig — client không biết route nào đến monolith, route nào đến service mới*

Tùy thuộc vào kiến trúc hiện tại, chúng ta có ba chiến lược triển khai Strangler Fig chính: **Route-based** (dùng Gateway định tuyến khác nhau đến monolith hoặc các service mới) rất phù hợp cho các hệ thống định hướng API (API-driven); **Event Interception** (service mới lắng nghe sự kiện từ monolith để dần thay thế logic) lý tưởng cho hệ thống định hướng sự kiện (event-driven); và **Asset Capture** (chuyển dần quyền sở hữu từng đối tượng nghiệp vụ cốt lõi — như một loại hồ sơ — sang hệ thống mới) thích hợp khi ranh giới dữ liệu rõ [W8].

Với KBM — nơi đã có API Gateway (Spring Cloud Gateway, Chương 8) — **route-based Strangler Fig** là lựa chọn tự nhiên nhất cho LMS chính. Với DevOps Lab, routing lại dựa trên hostname/wildcard DNS ở mức khái quát. Hai kiểu gateway này cùng tồn tại, cho thấy việc chuyển đổi không nhất thiết dùng một pattern duy nhất cho mọi bounded context.

Listing 10.1: *Cấu hình Spring Cloud Gateway áp dụng Strangler Fig (đường dẫn thi chuyển sang service mới, còn lại về monolith)*

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: new_exam_service_route
          uri: lb://kbm-exam
          order: 1
          predicates:
            - Path=/api/exams/**
        - id: legacy_monolith_fallback_route
          uri: http://kbm-legacy-monolith:8080
          order: 100
          predicates:
            - Path=/**
```

Chuyển đổi tăng dần

"If you do a big-bang rewrite, the only thing you're guaranteed of is a big bang."

— Martin Fowler, trích trong Sam Newman, Monolith to Microservices [4b, Ch.2]

10.2.4. KBM: Strangler Fig thực tế — từ `kbm-legacy-monolith` đến `kbm-*`

Trạng thái ban đầu (Before) của KBM (đã giới thiệu ở §1.8) là mô hình *independent silo monoliths* — ba khối MVC rời rạc chạy cạnh nhau mà không chia sẻ dữ liệu hay danh tính: phần quản lý đào tạo là ứng dụng **ASP.NET MVC** nguyên khối (`kbm-legacy-monolith`), còn các bộ chấm là những monolith MVC **tách rời** do nhóm tự viết — một code judge (C/C++ và Java, **không sandbox**) và một network judge (Java). Vì mỗi hệ là một khối riêng, chi phí bảo trì nhân lên theo *số hệ thống* chứ không được chia nhỏ.

Thay vì viết lại cả ba khối cùng một lúc ("Big Bang"), KBM tiến hóa tăng dần (2024–2025): các capability mới như SQL Practice (`kbm-judge-sql`), xác thực tập trung (`kbm-auth`), quản lý học vụ (`kbm-academic`), chấm mạng thế hệ mới (`kbm-judge-network`) lần lượt được phát triển thành các dịch vụ `kbm-*` đứng sau API Gateway (Spring Cloud Gateway, Chương 8). Cùng cơ chế điểm tiếp nối (seam) ở Gateway theo hướng định tuyến (`route-based` như Listing 10.1, với route dự phòng trỏ về `kbm-core` thay vì monolith cũ) tiếp tục được áp dụng khi cần bóc tách các phân hệ còn nằm trong `kbm-core` (chẳng hạn tách `kbm-exam` thành dịch vụ độc lập ở kiến trúc mục tiêu) — làm rõ ranh giới bounded context trước (Chương 2), rồi mới chuyển hướng định tuyến, đảm bảo mỗi bước đều nhỏ và có thể hoàn tác (rollback).

Xây → Mua: thay bộ chấm code tự viết bằng DOMjudge

Một bước Strangler Fig điển hình là `kbm-judge-code`. Bộ chấm mã nguồn (code judge) **Gen 1** (di sản, một monolith MVC riêng) do nhóm tự viết, chỉ hỗ trợ C/C++ và Java, và **chạy bài nộp trực tiếp trên môi trường OS của máy chủ — không có môi trường cách ly (sandbox)**: mã sinh viên có thể truy cập tài nguyên máy chủ, không giới hạn CPU/RAM. Đây vừa là lỗ hổng bảo mật (Chương 9) vừa là gánh nặng bảo trì.

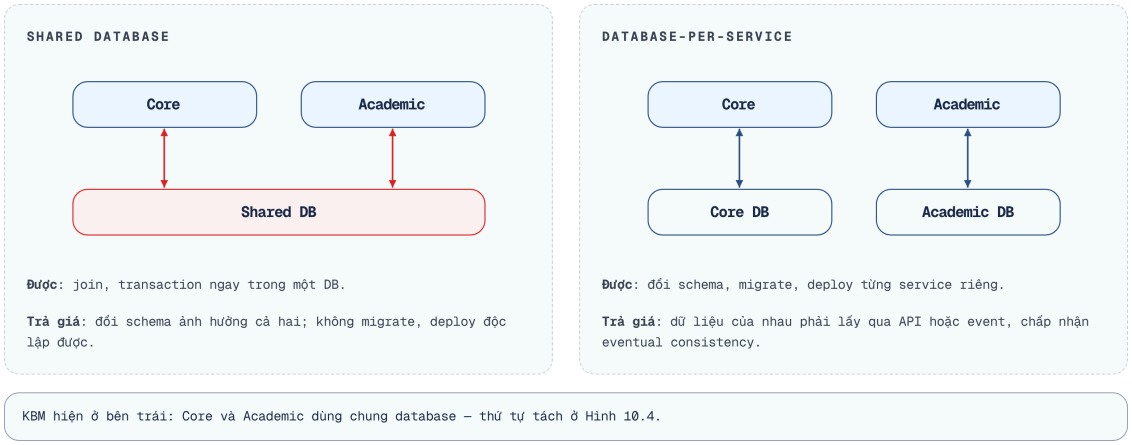
Thay vì tiếp tục chắp vá một bộ chấm tự xây, đội ngũ chọn **xây → mua/tích hợp (build → buy/adopt)**: `kbm-judge-code` **Gen 2** tích hợp **DOMjudge** — online judge mã nguồn mở chuẩn ICPC, đã

được tích hợp sẵn môi trường cách ly (sandbox bằng cgroup/namespace) và đa ngôn ngữ. Quyết định này **chuyển một lớp rủi ro ra khỏi nhóm**: cô lập sandbox và bảo trì compiler/runner được chuyển cho cộng đồng mã nguồn mở đã kiểm chứng; đội ngũ vẫn chịu trách nhiệm cấu hình, cập nhật và kiểm chứng môi trường judgehost khi tích hợp và vận hành. Đây là minh họa rằng việc chuyển đổi không phải lúc nào cũng là "tách ra rồi tự viết lại" — đôi khi bước hợp lý là **thay một thành phần tự xây bằng một sản phẩm đã được kiểm chứng**, loại bỏ một gánh nặng thay vì chỉ di dời nó.

10.3. Tách database — thách thức lớn

10.3.1. Vấn đề: database coupling nguy hiểm hơn code coupling

Tách mã nguồn (code) thành các dịch vụ (service) không quá khó — tách API, triển khai độc lập. Nhưng khi hai dịch vụ **chia sẻ cùng cơ sở dữ liệu** chúng vẫn bị ràng buộc (coupled) ở tầng sâu nhất:



Hình 10.3: Shared Database (coupled) vs Database-per-Service (decoupled)

Newman trong [4b, Ch.1] viết: **"One of the things I see people having the hardest time with is the idea that microservices should not share databases"** — và LMS là ví dụ trực tiếp: Core Service và Academic Service (`kbm-academic`) chia sẻ cùng database, những thay đổi về lược đồ (schema) ảnh hưởng đến cả hai, và hệ thống không thể triển khai các bản cập nhật cấu trúc cơ sở dữ liệu (database migration) một cách độc lập.

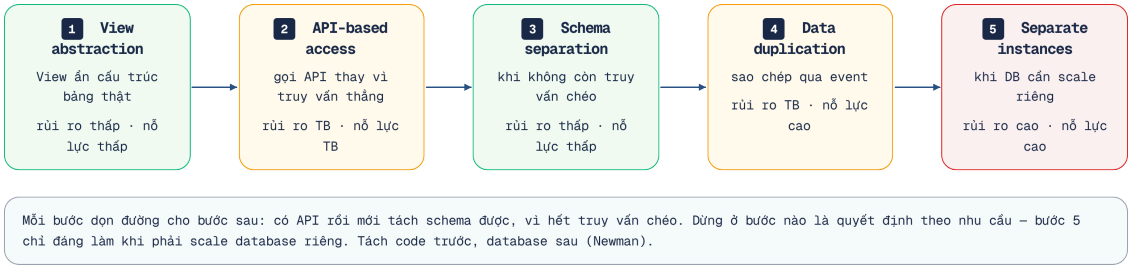
Từ Chương 7 (Database-per-Service), chúng ta đã biết danh mục năm chiến lược tách dữ liệu. Giờ hãy nhìn lại từ góc độ chuyển đổi — **thứ tự thực hiện** (kèm tên tương ứng đã học ở §7.2):

Bảng 10.3: 5 chiến lược tách database — từ góc độ chuyển đổi

#	Chiến lược (Strategy)	Mô tả	Rủi ro (Risk)	Nỗ lực (Effort)	Khi nào
1	**View abstraction** *(Database View, Ch.7)*	Tạo View ẩn đi cấu trúc bảng thực tế	● Thấp	Thấp	Bước đầu tiên — khi service B cần dữ liệu của service A
2	**API-based access** *(Database Wrapping Service, Ch.7)*	Service B gọi API service A thay vì truy vấn trực tiếp	● Trung bình	Trung bình	Khi thay thế việc gọi qua View/DB
3	**Schema separation** *(Separate Schema, Ch.7)*	Tách lược đồ (schema) trong cùng DB instance	● Thấp	Thấp	Khi không còn truy vấn chéo (nhờ API)

#	Chiến lược (Strategy)	Mô tả	Rủi ro (Risk)	Nỗ lực (Effort)	Khi nào
4	**Data duplication** *(Data Transfer, Ch.7)*	Sao chép dữ liệu thông qua các sự kiện (eventual consistency)	● Trung bình	Cao	Khi truy vấn chéo giữa các dịch vụ (cross-service query) trở nên phức tạp
5	**Separate instances** *(Full Database Split, Ch.7)*	Database instance riêng cho mỗi service	● Cao	Cao	Bước cuối — khi cần mở rộng cơ sở dữ liệu riêng

Sự dịch chuyển qua năm chiến lược trên được mô tả trong sơ đồ:



Hình 10.4: *Thứ tự thực hiện 5 chiến lược tách database*

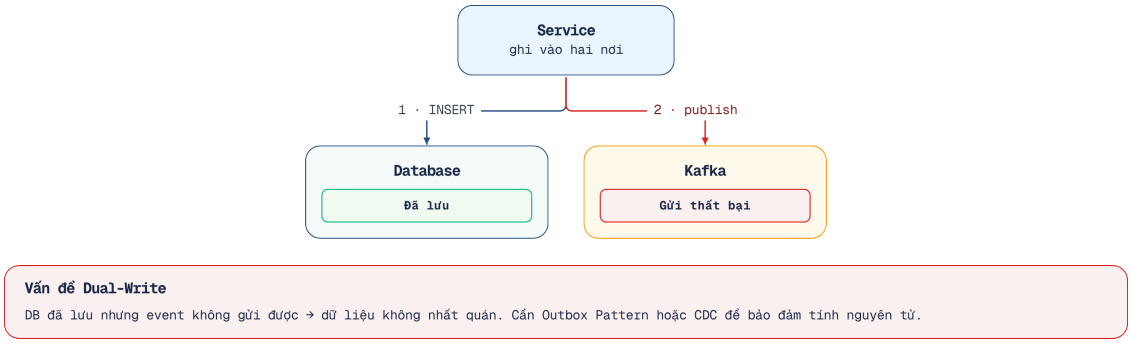
Tiến trình năm bước trên giúp giảm thiểu rủi ro và cho phép đội ngũ phát triển làm quen dần với độ phức tạp của dữ liệu phân tán. Đây là thứ tự tổng quát theo hướng *API trước*: gỡ truy vấn chéo rồi mới tách schema. Chương 7 đặt nó cạnh hướng *schema trước* — dựng ranh giới rẻ trong cùng DB server, tạm cho đọc chéo qua view rồi thay dần bằng API — và giải thích vì sao KBM, với hai service dùng chung một PostgreSQL và một nhóm nhỏ, chọn hướng sau (mục "Áp dụng cho KBM" ở §7.2). Lộ trình Giai đoạn 4 ở cuối chương đi theo lựa chọn đó.

Tách database tăng dần

> "We split both the code and data at once. [...] I strongly suggest avoiding this approach, and instead splitting either the schema or application tier first."
>
> — *Sam Newman, Monolith to Microservices [4b, Ch.4]*

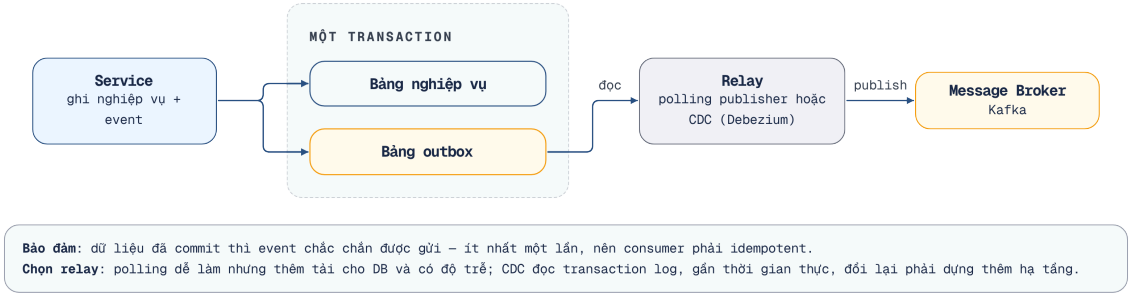
10.3.2. Cạm bẫy ghi kép (dual-write) và Outbox Pattern

Khi tách database, một anti-pattern phổ biến là **dual write**: dịch vụ vừa ghi vào cơ sở dữ liệu vừa phát tán sự kiện (publish event) — nếu một trong hai thất bại, dữ liệu sẽ không nhất quán.



Hình 10.5: *Cạm bẫy ghi kép — DB thành công nhưng event thất bại*

Trạng thái không nhất quán này đặc biệt nguy hiểm với việc nộp bài thi hoặc ghi nhận điểm số. Cách xử lý phổ biến là dùng chính database nội bộ làm điểm trung chuyển, tách thao tác ghi nghiệp vụ khỏi thao tác đẩy dữ liệu qua mạng. **Outbox Pattern** giải quyết vấn đề bằng cách ghi sự kiện vào bảng outbox *cùng một giao dịch* với dữ liệu nghiệp vụ chính; sau đó, một tiến trình ngầm sẽ đọc bảng này và đẩy thông điệp lên message broker:



Hình 10.6: *Outbox Pattern — sự kiện được ghi cùng giao dịch với dữ liệu nghiệp vụ*

Có hai cách để tiến trình ngầm đọc và đẩy sự kiện từ bảng outbox. Phương pháp thứ nhất là **Polling publisher**, sử dụng một luồng chạy ngầm để liên tục kiểm tra bảng outbox; cách này dễ triển khai nhưng tạo thêm tải cho cơ sở dữ liệu và có độ trễ nhất định. Phương pháp thứ hai tiên tiến hơn là sử dụng **CDC (Change Data Capture)** như Debezium để đọc trực tiếp từ transaction log; cách này cho độ trễ thấp và giảm tải polling (độ trễ thực tế phụ thuộc tải và vận hành, cần đo lag), nhưng đòi hỏi phải thiết lập hạ tầng phức tạp hơn.

Outbox Pattern

Giống như sinh viên nộp bài thi (business data) và ký tên vào danh sách điểm danh (event) *cùng một lúc* tại bàn giáo viên. Sau đó, một giám thị khác (tiến trình ngầm) mới cầm danh sách điểm danh đó báo cáo cho Phòng Đào tạo (Message Broker).

Để bài làm của sinh viên không bị mất khi sự cố mạng xảy ra giữa lúc ghi dữ liệu và phát sự kiện, dưới đây là cách cài đặt Outbox Pattern vào hệ thống nộp bài:

Listing 10.2: *Mã nguồn Transactional Outbox Pattern (lưu cả hai thực thể trong một giao dịch)*

```
@Service
public class SubmissionService {
    private final SubmissionRepository submissionRepo;
    private final OutboxRepository outboxRepo;

    public SubmissionService(SubmissionRepository submissionRepo,
                             OutboxRepository outboxRepo) {
        this.submissionRepo = submissionRepo;
        this.outboxRepo = outboxRepo;
    }

    @Transactional
    public void submitExam(Submission submission) {
        // 1. Lưu business data
        submissionRepo.save(submission);
```

```
        // 2. Lưu event vào bảng Outbox (cùng transaction)
        OutboxEvent event = new OutboxEvent(
            "SubmissionCreated",
            toJson(submission)
        );
        outboxRepo.save(event);
    }

    // toJson(...): hàm phụ chuyển Submission sang JSON, lược bớt
}
```

KBM chưa có Outbox Pattern

KBM hiện dùng **dual write**: Core Service lưu bài nộp vào DB rồi gọi `submitProducer.sendSubmission()` đến Kafka. Nếu Kafka không khả dụng (unavailable) tại thời điểm đó, bài nộp (submission) đã lưu nhưng Judge Service không nhận được bài — sinh viên thấy "đã nộp" nhưng không nhận kết quả chấm.

Lộ trình chuyển đổi: (1) Ngắn hạn — cơ chế thử lại (retry logic) cho Kafka producer (đã có nhưng cần được kiểm chứng lại một cách kỹ lưỡng), (2) Trung hạn — bảng outbox + polling publisher, (3) Dài hạn — Debezium CDC. Với lưu lượng truy cập (traffic) thấp của KBM hiện tại, polling publisher (trung hạn) đủ tốt — Debezium chỉ cần khi mở rộng quy mô trên môi trường thực tế lớn.

10.3.3. CDC cho chuyển đổi dữ liệu

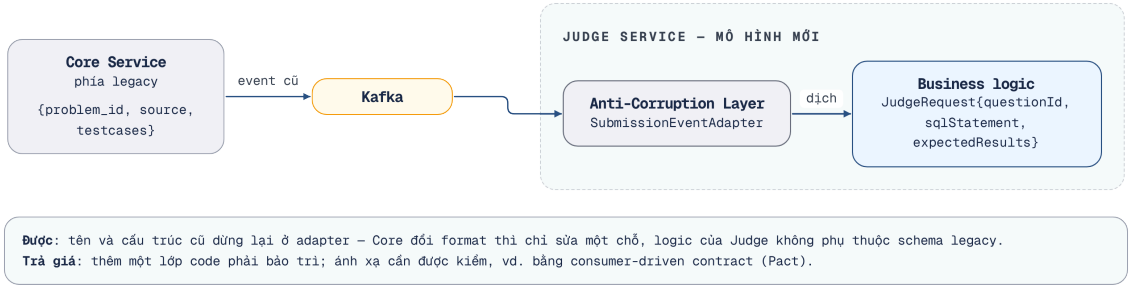
Lưu ý quan trọng: Trong chương này, chúng ta mới nhắc CDC trong ngữ cảnh Outbox Pattern. Tuy nhiên, CDC (như Debezium) thực chất là công cụ lõi để **đồng bộ dữ liệu thời gian thực (Real-time Sync)** khi phân tách cơ sở dữ liệu. Monolith vẫn tiếp tục ghi vào DB cũ, CDC sẽ nắm bắt (capture) các thay đổi đó và đẩy sang DB của Microservice mới liên tục. Nhờ vậy, cửa sổ gián đoạn khi chuyển đổi (cut-over) được rút xuống rất nhỏ (near-zero downtime): chỉ cần chặn thao tác ghi vài giây để CDC đuổi kịp độ trễ sao chép (replication) rồi mới chuyển hướng. Zero downtime tuyệt đối đòi hỏi thêm dual-write và cơ chế xử lý xung đột — chi phí thường không tương xứng ở quy mô KBM.

10.4. Anti-Corruption Layer và các pattern chuyển đổi

Khi dịch vụ mới bắt buộc phải gọi đến hệ thống di sản (legacy), chúng ta cần một lớp đệm trung gian để chuyển đổi mô hình dữ liệu, ngăn chặn thiết kế lạc hậu của hệ thống cũ ảnh hưởng đến kiến trúc mới.

10.4.1. Anti-Corruption Layer (ACL) — bảo vệ ranh giới

Khi tách dịch vụ mới ra khỏi monolith, dịch vụ mới cần giao tiếp với mã nguồn di sản (legacy) — nhưng **không nên để mô hình (model) cũ ảnh hưởng tiêu cực đến mã nguồn mới**. Eric Evans trong [6] định nghĩa **Anti-Corruption Layer** (ACL): một lớp dịch ngôn ngữ giữa hai bounded context có mô hình (model) khác nhau.



Hình 10.7: *Anti-Corruption Layer — lớp dịch giữa service mới và legacy*

Ví dụ minh họa với KBM: Judge Service nhận các bài nộp (submission) từ Core Service qua Kafka. Giả sử Core Service còn truyền dữ liệu theo định dạng cũ `{problem_id, source, testcases}`. Judge Service có thể: (1) × dùng trực tiếp tên trường dữ liệu di sản (legacy field name) trong mã nguồn — bị ràng buộc (coupling) với các quyết định của hệ thống di sản, hoặc (2) √ tạo adapter dịch sang mô hình nội bộ `JudgeRequest{questionId, sqlStatement, expectedResults}` — ranh giới rõ ràng (clean boundary).

ACL hoạt động như một "Thông dịch viên" hoặc hệ thống quy đổi tín chỉ: nó giúp dịch vụ mới tích hợp với hệ thống cũ (Monolith) mà không bị xung đột dữ liệu vì khác biệt mô hình miền (Domain Model).

Đoạn mã dưới đây minh họa cách một Adapter thực hiện chuyển đổi luồng dữ liệu một cách trong suốt:

Listing 10.3: *Adapter trong Anti-Corruption Layer dịch sự kiện cũ (legacy event) sang mô hình mới*

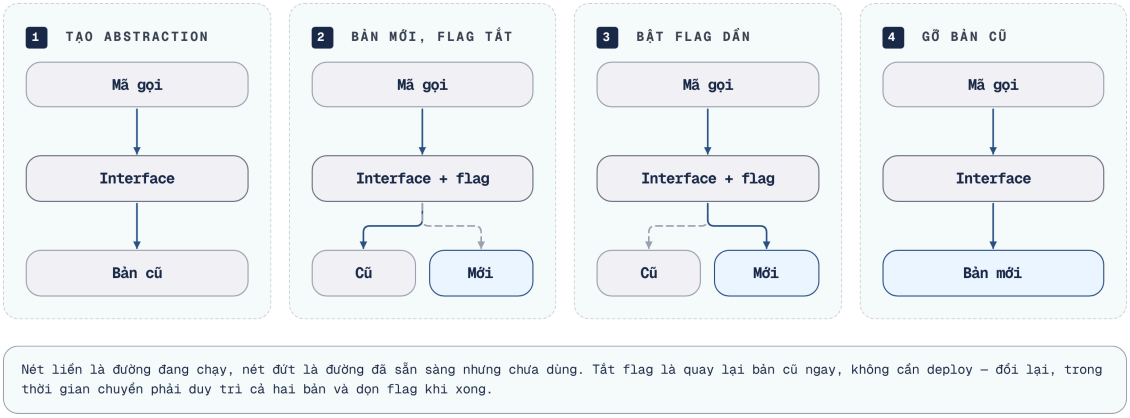
```
@Component
public class SubmissionEventAdapter {
    // Dịch từ model cũ (Monolith) sang model mới (Judge Service)
    public JudgeRequest translate(LegacySubmissionEvent legacyEvent) {
        return new JudgeRequest(
            legacyEvent.getProblemId(),
            legacyEvent.getSource(),
            parseTestcases(legacyEvent.getTestcases()) // Phân tích định dạng cũ
        );
    }

    // parseTestcases(...): hàm phụ phân tích định dạng cũ, lược bớt
}
```

Khi bóc tách service, để đảm bảo việc thay đổi ở service mới không vô tình phá vỡ monolith cũ đang gọi nó, chúng ta nên áp dụng **Consumer-Driven Contracts** (ví dụ framework Pact) để tự động hóa việc kiểm tra tính tương thích API giữa hai hệ thống.

10.4.2. Branch by Abstraction

Newman trong [4b, Ch.3] đề xuất **Branch by Abstraction** cho việc chuyển đổi an toàn:



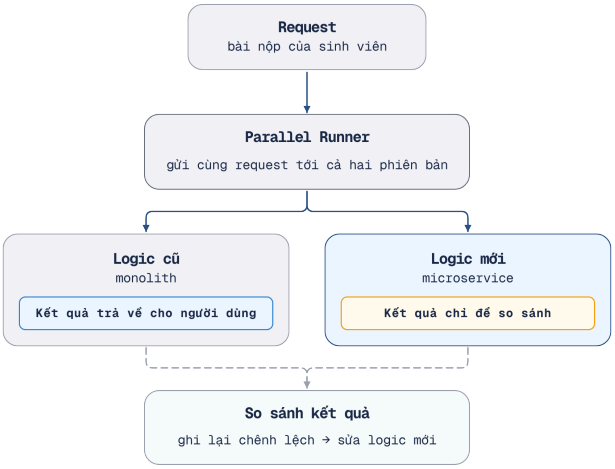
Hình 10.8: Branch by Abstraction — chuyển đổi an toàn qua feature flag

1. **Tạo lớp trừu tượng** (abstraction layer, interface) che chắn cho phần hiện thực cũ
2. **Hiện thực hóa phiên bản mới** phía sau cùng interface, kiểm soát bằng feature flag
3. **Chuyển lưu lượng** dần sang phiên bản mới và kiểm chứng
4. **Gỡ bỏ** phiên bản cũ khi đã chắc chắn về tính ổn định

Để quản lý Feature Flag chuyên nghiệp trong Java, thay vì dùng `if-else` cứng và cấu hình properties, chúng ta nên dùng các thư viện chuyên dụng như **Togglz**, **Unleash**, hoặc **FF4J** / **OpenFeature**. Các thư viện này cho phép bật/tắt luồng logic ngay tại runtime mà không cần khởi động lại dịch vụ.

10.4.3. Parallel Run — kiểm chứng trước khi chuyển đổi

Parallel Run — chiến lược cho phép **chạy đồng thời** cả phiên bản hiện thực cũ và mới, sau đó đối chiếu kết quả đầu ra. Ở cấp độ hạ tầng, API Gateway có thể áp dụng **Traffic Shadowing (Dark Launching)**: nhân bản âm thầm (shadow) một luồng lưu lượng (traffic) thực tế và chuyển hướng đến dịch vụ mới để kiểm tra tải và kiểm tra logic mà người dùng không hề hay biết.



Hình 10.9: Parallel Run — chạy song song logic cũ và mới, so sánh kết quả

- Phiên bản cũ **trả kết quả (response)** cho người dùng (đảm bảo an toàn — production-safe)

- Phiên bản mới chạy **song song** — kết quả trả về bị loại bỏ (discard)
- So sánh hai kết quả — ghi nhận (log) các điểm sai lệch (mismatch)
- Khi tỷ lệ sai lệch (mismatch rate) ≈ 0% → chuyển đổi (switch) sang phiên bản mới

⚠ Parallel Run chỉ an toàn với thao tác đọc

Với thao tác ghi (INSERT, UPDATE), chạy song song sẽ tạo tác dụng phụ kép (ghi hai lần). Để kiểm chứng an toàn, hãy dùng kỹ thuật **Shadow Database** hoặc **Discard Writes** — loại bỏ mọi thao tác ghi của phiên bản mới.

Áp dụng cho KBM: nếu cần viết lại `CompareUtil` (logic so sánh kết quả SQL — quan trọng nhất trong hệ thống chấm điểm), Parallel Run cho phép chạy phiên bản `CompareUtil` cũ và mới song song trên mỗi bài nộp, so sánh kết quả để đảm bảo logic chấm điểm mới hoàn toàn khớp trước khi chính thức chuyển đổi. Tuy nhiên, hệ thống phải hủy bỏ (discard) các thao tác ghi (write/side-effect) của phiên bản mới để tránh làm hỏng dữ liệu thật.

Chiến lược Parallel Run giúp xác minh tính chính xác của phiên bản mới bằng dữ liệu thực tế (production data) trước khi chuyển hướng hoàn toàn lưu lượng truy cập (cut-over).

Chuyển đổi phải hoàn tác được

Mỗi bước chuyển đổi phải hoàn tác được. Nếu không quay lui dễ dàng, nghĩa là đang nhận quá nhiều rủi ro một lần. Feature flag, parallel run và định tuyến tăng dần đều phục vụ cùng một mục đích: *làm cho việc thất bại trở nên an toàn.*

— Theo tinh thần Sam Newman, Monolith to Microservices [4b, Ch.2–3]

10.5. Quá trình chuyển đổi kiến trúc tại KBM

Phần này tổng hợp các khoảng trống đã phân tích từ Chương 2–9, đồng thời xem trước (preview) hai điều kiện tiên quyết về khả năng quan sát (Chương 11) và tự động hóa triển khai (Chương 12) thành một danh mục duy nhất, rồi sắp xếp thành lộ trình chuyển đổi bốn giai đoạn ưu tiên theo Impact × Effort.

10.5.1. Tổng hợp các phân tích khoảng trống

Từ Chương 2–9, chúng ta đã phân tích khoảng trống (gap) giữa hiện trạng KBM tại thời điểm khảo sát (đầu năm 2026) và các giải pháp khuyến nghị. Tuy nhiên, thực tế chuyển đổi cho thấy không thể tách database an toàn nếu hệ thống chưa có khả năng quan sát và quy trình triển khai tự động — hai chủ đề sẽ được học chi tiết ngay ở Chương 11 và Chương 12. Vì vậy, Bảng 10.4 gom các khoảng trống từ Chương 2–9 kèm hai điều kiện tiên quyết xem trước của Chương 11–12 (bức tranh tích hợp toàn cục sẽ được nhìn lại ở phần Kết luận):

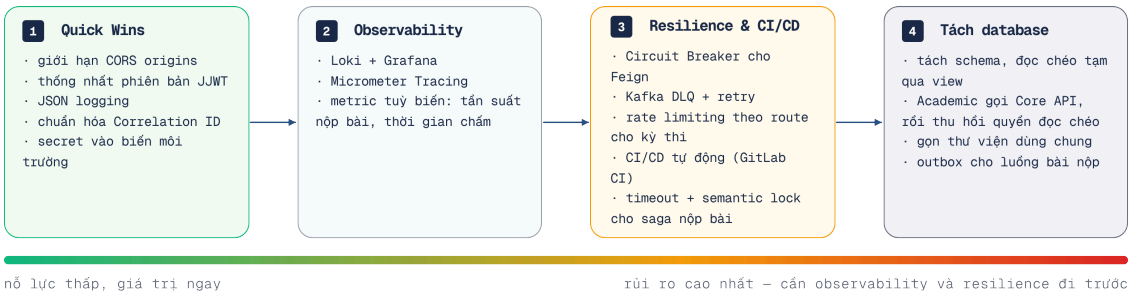
Bảng 10.4: *Tổng hợp khoảng trống của KBM tại thời điểm khảo sát (Ch.2–9) và hai điều kiện tiên quyết xem trước (Ch.11–12)*

Chương	Khoảng trống	Mức độ	Tham chiếu
Ch.2	7 core bounded context rõ hơn nhưng thư viện dùng chung vẫn gây coupling	● Trung bình	§2.5
Ch.3	Đặt tên/phiên bản API chưa nhất quán; contract worker của SQL Judge cần chuẩn hóa	● Trung bình	§3.9

Chương	Khoảng trống	Mức độ	Tham chiếu
Ch.4	SQL Judge coordinator/worker và tích hợp bên ngoài cần ACL/resilience rõ hơn	● Trung bình	§4.5
Ch.5	Kafka pipeline cần DLQ/idempotency; SQL Judge dispatch và DevOps Lab queue cần chọn broker theo khối lượng công việc (workload)	● Trung bình	§5.9
Ch.6	Saga nộp bài ngầm định: thiếu timeout, compensation, semantic lock	● Trung bình	§6.6
Ch.7	**Shared database** (Core ↔ Academic)	● Cao	§7.6
Ch.8	Spring Cloud Gateway + Go router cần thống nhất chính sách ở biên	● Cao	§8.5
Ch.9	HS256 shared secret, lưu trữ token, quản lý secret, SSO đa nền tảng	● Cao	§9.6
Ch.11 *(xem trước)*	Observability có Actuator/Prometheus một phần, chưa có truy vết đầu cuối và log tập trung	● Cao	§11.9
Ch.12 *(xem trước)*	LMS chính dùng Docker Compose; DevOps Lab dùng k3s/Sysbox; CI/CD và rollback chưa đồng đều	● Cao	§12.7

10.5.2. Lộ trình chuyển đổi — 4 giai đoạn

Nguyên tắc ưu tiên: **Quick Wins trước, các thay đổi rủi ro cao (High-Risk) thực hiện sau**. Mỗi giai đoạn mang lại giá trị ngay lập tức — không cần đợi hoàn thành toàn bộ.



Hình 10.10: *Lộ trình chuyển đổi 4 giai đoạn — Quick Wins → Observability → Resilience → Decomposition*

Việc chia nhỏ thành bốn giai đoạn giúp đội ngũ ưu tiên giải quyết các vấn đề có tác động cao nhưng đòi hỏi nỗ lực thấp (High Impact / Low Effort) trước, và hoãn lại các thay đổi cơ sở dữ liệu rủi ro cao cho đến khi có đủ cơ chế giám sát.

Giai đoạn 1 — Các cải tiến nhanh (Quick Wins): nỗ lực thấp, tác động cao

Giai đoạn mở đầu tập trung vào những thay đổi nhỏ, tốn ít công sức nhưng mang lại hiệu quả bảo mật và vận hành đáng kể, không đòi hỏi thay đổi kiến trúc cốt lõi.

Bảng 10.5: *Giai đoạn 1 — Quick Wins*

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Giới hạn CORS origin	Ch.8	30 phút	Thu hẹp chính sách CORS quá rộng
2	Thống nhất phiên bản JJWT trong parent POM	Ch.8	1 giờ	Ngăn lỗi sai lệch phiên bản
3	Chuyển sang JSON logging (Logback encoder)	Ch.11	2 giờ	Log có thể tìm kiếm

#	Hành động	Khoảng trống	Nỗ lực	Tác động
4	Chuẩn hóa ngữ cảnh truy vết: `X-Request-ID` ở HTTP, `traceparent` và `correlationId` qua Kafka	Ch.8, Ch.11	4 giờ	Gỡ lỗi xuyên dịch vụ và luồng bất đồng bộ
5	Đưa thông tin nhạy cảm ra khỏi tệp cấu hình, nạp qua biến môi trường hoặc kho secret; luân phiên secret định kỳ	Ch.9	2 giờ	Giảm nguy cơ lộ secret

Triết lý Giai đoạn 1: Không thay đổi kiến trúc tổng thể, không can thiệp sâu vào logic cốt lõi — chỉ tập trung **chuẩn hóa cấu hình và tăng cường bảo mật**. Một kỹ sư duy nhất cũng đủ sức hoàn thành nhóm công việc này trong một đến hai sprint — và đây là bước đệm cần thiết trước khi bắt tay vào các phần phân tách phức tạp hơn.

Giai đoạn 2 — Nền tảng Observability

Sau Giai đoạn 1, hệ thống cần được trang bị khả năng giám sát đầy đủ trước khi tiến hành phân tách các thành phần phức tạp.

Bảng 10.6: *Giai đoạn 2 — Nền tảng Observability*

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Triển khai Loki + Grafana	Ch.11	1–2 ngày	Tra cứu log tập trung
2	Thêm Micrometer Tracing	Ch.11	1 ngày	Biết nút thắt cổ chai (bottleneck) ở đâu
3	Metric tùy biến (tần suất nộp bài, thời gian chấm)	Ch.11	2 ngày	Giám sát chủ động (Proactive monitoring)

Triết lý Giai đoạn 2: Trước khi thực hiện các thay đổi kiến trúc phức tạp ở Giai đoạn 3 và 4, hệ thống cần có cơ chế giám sát tốt. Dashboard và hệ thống log tập trung là công cụ quan trọng để gỡ lỗi.

Giai đoạn 3 — Resilience & CI/CD

Với khả năng giám sát đã sẵn sàng, bước tiếp theo là xây dựng cơ chế phòng thủ để ứng dụng chịu được lỗi và tự động hóa quy trình phân phối.

Bảng 10.7: *Giai đoạn 3 — Resilience & CI/CD*

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Resilience4j Circuit Breaker cho Feign client	Ch.4	2–3 ngày	Ngăn chặn lỗi dây chuyền (cascading failure)
2	Kafka Dead Letter Queue + retry	Ch.5	2 ngày	Giảm thất lạc message đã vào broker; khoảng trống DB–broker cần Outbox (§10.3)
3	Áp chính sách rate limiting theo route cho kỳ thi (trên token bucket Redis đã có)	Ch.8	1–2 ngày	Bảo vệ hệ thống khi có lưu lượng lạm dụng
4	Hoàn thiện CI/CD triển khai tự động (GitLab CI)	Ch.12	3–5 ngày	Tự động hóa biên dịch và triển khai
5	Timeout + semantic lock cho saga nộp bài	Ch.6	2–3 ngày	Bài nộp không treo ở PENDING

Triết lý Giai đoạn 3: Hệ thống phải chứng minh được **tính bền bỉ trước khi bị xẻ nhỏ**. Việc phân tách cơ sở dữ liệu khi chưa có cơ chế ngắt mạch (circuit breaker) hay hệ thống cảnh báo sẽ dễ gây ra lỗi dây chuyền một khi có bất kỳ dịch vụ nào gặp sự cố nghẽn mạng.

Giai đoạn 4 — Phân tách cơ sở dữ liệu (database decomposition): thận trọng nhất
Bước cuối cùng — tháo gỡ sự phụ thuộc ở tầng dữ liệu — là thử thách lớn, và đòi hỏi sự ổn định từ ba giai đoạn trước.

__Bảng 10.8:__ Giai đoạn 4 — Phân tách cơ sở dữ liệu

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Schema separation (`core_schema`, `academic_schema`); truy vấn chéo còn lại tạm đọc qua view	Ch.7	1–2 tuần	Ranh giới thấy được, đếm được truy vấn chéo
2	API-based access (Academic gọi Core API); hết truy vấn chéo thì thu hồi quyền đọc chéo	Ch.7	2–3 tuần	Loại bỏ truy cập DB trực tiếp
3	Refactor thư viện dùng chung (chỉ giữ DTO, ngoại lệ)	Ch.2	2–3 tuần	Giảm thiểu sự phụ thuộc (coupling)
4	Outbox pattern cho luồng xử lý bài nộp	Ch.5, Ch.6	1–2 tuần	Giao tiếp tin cậy (Reliable messaging)

Triết lý Giai đoạn 4: Đây là tập hợp những thay đổi chứa đựng **nhiều rủi ro nhất** trong toàn bộ lộ trình. Chúng ta dựa vào nền tảng quan sát (Giai đoạn 2) và tính bền bỉ (Giai đoạn 3) để thực hiện bước này. Mỗi thay đổi cần được triển khai từ từ, theo dõi qua dashboard, khi ổn định mới tiến hành bước tiếp theo.

10.5.3. Ma trận quyết định — ưu tiên theo tác động × nỗ lực

Để chọn việc làm trước trong lộ trình, Hình 10.11 xếp các hạng mục theo hai trục: tác động và nỗ lực.



__Hình 10.11:__ Ma trận quyết định — ưu tiên theo tác động × nỗ lực

Chuyển đổi là cuộc chạy đường dài

Đừng cố gắng khắc phục mọi khoảng trống cùng lúc. Trong mỗi chu kỳ phát triển (sprint), chỉ nên ưu tiên 1–2 hạng mục thuộc nhóm “Tác động cao, nỗ lực thấp (High Impact, Low Effort)” trước. Khi đã hết các thành quả nhanh (quick wins), mới chuyển sang nhóm “Tác động cao, nỗ lực cao” — và luôn phải đảm bảo khả năng quan sát (observability) trước khi thực hiện các thay đổi lớn.

10.6. Sai lầm thường gặp khi chuyển đổi

Ngoài 4 anti-pattern ở Bảng 10.2 (§10.1), quá trình chuyển đổi còn có những rủi ro cần được nhận diện và xử lý sớm:

- **Distributed Monolith**: Tách các dịch vụ (service) nhưng vẫn dùng chung cơ sở dữ liệu mà không làm rõ ranh giới sở hữu, dẫn đến quy trình triển khai phải đồng bộ. Newman trong [4b, Ch.1] nhận xét loại hệ thống này "have all the disadvantages of a distributed system, and the disadvantages of a single-process monolith, without having enough upsides of either". *Phòng tránh* bằng cách xác lập rõ quyền sở hữu từng bảng (logical ownership), tách schema/quyền truy cập và thay truy vấn chéo bằng API trước khi tách cơ sở dữ liệu vật lý theo nhu cầu mở rộng.
- **Big Bang Rewrite**: Viết lại toàn bộ từ đầu (from scratch), ngày chuyển đổi ("switch") không bao giờ đến. *Phòng tránh* bằng cách sử dụng Strangler Fig (§10.2) — mỗi chu kỳ phát triển (sprint) tách một phần, hệ thống luôn ở trạng thái sẵn sàng triển khai (deployable).
- **Bỏ qua Conway's Law**: Phân tách thành các dịch vụ độc lập nhưng cấu trúc đội ngũ phát triển vẫn như cũ. *Phòng tránh* bằng cách tổ chức lại đội ngũ theo các bounded context (Chương 2) — mỗi nhóm chỉ nên sở hữu 1–2 dịch vụ.
- **"Lift and Shift" không thiết kế lại**: Sao chép nguyên bản mã nguồn vào các container, và ngộ nhận đó là kiến trúc microservices. *Phòng tránh* bằng cách sử dụng Anti-Corruption Layer (§10.4) ngăn chặn mô hình dữ liệu cũ (legacy model) xâm nhập vào kiến trúc mới.
- **Hạ tầng phức tạp quá mức cần thiết**: Triển khai K8s + Istio cho 3 dịch vụ với 100 yêu cầu/phút. *Phòng tránh* bằng cách bắt đầu đơn giản (Docker Compose, xem trước Chương 12), chỉ mở rộng quy mô khi lưu lượng truy cập thực sự đòi hỏi.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Chiến lược Strangler Fig Pattern**: strangler-fig-migration.html
- **Transactional Outbox**: outbox-pattern.html
- **CDC và Outbox**: cdc-outbox.html

10.7. Tổng kết

Chuyển đổi từ monolith sang microservices không phải quyết định kỹ thuật thuần túy — đây là sự kết hợp giữa **tổ chức đội ngũ (team organization)** (Conway's Law, Chương 2), **hiểu biết về miền nghiệp vụ (domain understanding)** (Bounded Context, Chương 2), **các mô hình giao tiếp (communication pattern)** (Chương 3–6), **chiến lược dữ liệu (data strategy)** (Chương 7), **sự sẵn sàng của hạ tầng biên và bảo mật** (Chương 8–9), và **mức độ trưởng thành trong vận hành** (sẽ học ở Chương 11–12).

Bài học quan trọng nhất: **"Monolith First"** — bắt đầu đơn giản, chỉ chuyển đổi (migrate) khi có lý do cụ thể, thực hiện chuyển đổi tăng dần (Strangler Fig), và mỗi bước đều phải có khả năng hoàn tác (reversible). Việc đập đi xây lại toàn bộ (Big Bang rewrite) gần như luôn thất bại; Strangler Fig giảm rủi ro đáng kể — vì nó cho phép chấp nhận các rủi ro ở quy mô nhỏ, rút kinh nghiệm nhanh chóng và liên tục mang lại giá trị thực tiễn.

Tách database là một trong những thách thức lớn nhất — sự phụ thuộc về mã nguồn có thể giải quyết qua tái cấu trúc (refactoring), nhưng sự phụ thuộc về dữ liệu đòi hỏi một chiến lược chuyển

đổi thận trọng: đi theo tiến trình tổng quát *API trước* (view abstraction → API-based access → schema separation → data duplication → separate instances) hoặc lộ trình *schema trước* khi nhóm nhỏ dùng chung một DB server như KBM (tách schema để lộ truy vấn chéo, thay dần bằng API rồi chỉ tách DB vật lý khi thực sự cần mở rộng riêng). Outbox Pattern giải quyết bài toán giao tiếp tin cậy (reliable messaging) trong quá trình chuyển đổi — giúp tránh cái bẫy ghi dữ liệu kép (dual-write pitfall).

Anti-Corruption Layer, Branch by Abstraction, và Parallel Run là bộ công cụ cho việc chuyển đổi an toàn — mỗi pattern phục vụ mục đích khác nhau nhưng chia sẻ nguyên tắc chung: **làm cho quá trình chuyển đổi có thể đảo ngược, chia nhỏ từng bước tiến hành, và đảm bảo hệ thống luôn trong trạng thái sẵn sàng triển khai**.

Lộ trình chuyển đổi cho KBM — tổng hợp từ các khoảng trống ở Chương 2–9 cùng hai điều kiện tiên quyết xem trước ở Chương 11–12 — cho thấy: các Quick Wins như CORS, JWT, log có cấu trúc (structured logging) có thể hoàn thành trong 1–2 tuần, mang lại giá trị ngay. Phân tách cơ sở dữ liệu (Database decomposition) — thay đổi lớn nhất — cần được thiết kế dựa trên nền tảng về khả năng quan sát (observability) và tính bền bỉ (resilience). DevOps Lab cho thấy cùng một hệ thống có thể cần chiến lược khác nhau theo bounded context: Docker Compose đủ tốt cho LMS chính, trong khi việc cô lập phòng thí nghiệm cần k3s/Sysbox. Mỗi quyết định thiết kế đều là một sự đánh đổi — không có phương án "đúng tuyệt đối", chỉ có phương án "phù hợp nhất với bối cảnh hệ thống tại thời điểm đó".

Ở Chương 11, chúng ta sẽ xem cách **quan sát và giám sát** hệ thống sau khi chuyển đổi — logging, metric, tracing — ba kỹ thuật trọng tâm giúp phát hiện và xử lý vấn đề trước khi người dùng bị ảnh hưởng.

10.8. Đọc thêm

Sách tham khảo chính:

1. [4b] Sam Newman, *Monolith to Microservices* — Toàn bộ sách: decomposition patterns, database splitting, migration strategies
2. [2a] Chris Richardson, *Microservices Patterns* 1st Ed. — Ch.13: Refactoring to Microservices — Strangler Fig, Anti-Corruption Layer
3. [4a] Sam Newman, *Building Microservices* — Ch.3: How to Model Services (decomposition), Ch.5: Splitting the Monolith

Sách bổ trợ:

4. [6] Eric Evans, *Domain-Driven Design* — Ch.14: Anti-Corruption Layer, Context Mapping
5. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.3: Designing Microservices (SEED(S))
6. [2b] Chris Richardson, *Microservices Patterns* 2nd Ed. — Ch.2: Migration Anti-patterns; Ch.21: Refactoring a monolith to microservices (Strangler Fig, cập nhật)

Nguồn trực tuyến:

- [W2] Martin Fowler, "MonolithFirst" — martinfowler.com/bliki/MonolithFirst.html
- [W8] Martin Fowler, "StranglerFigApplication" — martinfowler.com/bliki/StranglerFigApplication.html
- Martin Fowler, "MicroservicePremium" — martinfowler.com/bliki/MicroservicePremium.html

- Sam Newman, “Breaking Apart the Monolith” — samnewman.io
- Debezium (CDC) — debezium.io
- Martin Fowler, “BranchByAbstraction” — martinfowler.com/bliki/BranchByAbstraction.html

Bài tập: 10-0, 10-1, 10-2, 10-3 ở phần Bài tập & case study cuối sách.

Tài liệu tham khảo

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[2b] Richardson, C. (2025). *Microservices Patterns* (2nd ed.; bản MEAP, truy cập 07/2026). Manning Publications.

[3] Mitra, R., & Nadareishvili, I. (2020). *Microservices: Up and Running*. O’Reilly Media.

[4a] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media.

[4b] Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O’Reilly Media.

[6] Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.